Tensor.h & Tensor.c

Complete Usage Guide — every concept defined, every function explained with why & how

1 — Technical Concepts Glossary

Before reading any code, make sure these fundamental terms are clear. Every one of them appears repeatedly in Tensor.h and Tensor.c.

struct: A container that groups related variables together under one name. Instead of having separate variables for every field, you bundle them into one named group.
Analogy: A passport — it groups your name, date of birth, nationality, and photo in one document.

typedef: Gives an existing type a new, shorter name. typedef struct MyStruct { ... } myStruct_t; means you can write myStruct_t instead of struct MyStruct.
Analogy: Calling a 'mobile telephone' just a 'phone' — shorter, same thing.

pointer (*): A variable that stores a memory address, not a value directly. Instead of holding the actual data, it holds the location where the data lives.
Analogy: A sticky note with a room number written on it. The note is not the room — it tells you where to go.

NULL pointer: A pointer that points to nothing — address 0. Used to mean 'this field is not set' or 'this feature is not active'.
Analogy: A sticky note with nothing written on it — it does not point anywhere.

uint8_t: An unsigned 8-bit integer. Values 0–255. One byte. Used as the raw storage type for all tensor data regardless of the actual number format.
Analogy: A single box that can hold a number between 0 and 255.

size_t: An unsigned integer whose size matches the platform — 4 bytes on STM32, 8 bytes on a 64-bit PC. Used for array sizes, byte counts, and indices.
Analogy: A ruler that changes length depending on what country you are in.

enum: A list of named integer constants. enum qtype { INT32=0, FLOAT32=1, ... } lets you write INT32 instead of the number 0.
Analogy: A menu where each item has a number behind the scenes, but you order by name.

void *: A pointer to an unknown type. The compiler accepts any type of data at this address. You must cast it back to the correct type before using it.
Analogy: A USB port — it accepts any device, but the device needs its own driver to work.

bitwise AND &: Compares two numbers bit by bit. Result bit is 1 only if BOTH input bits are 1. Used to isolate specific bits or clear specific bits.
Analogy: Two people must BOTH say yes for a decision to be yes.

bitwise OR |: Compares two numbers bit by bit. Result bit is 1 if EITHER input bit is 1. Used to set specific bits without changing others.
Analogy: Either person saying yes is enough.

bit shift >> <<: >> moves all bits to the right (divides by 2 per shift). << moves all bits to the left (multiplies by 2 per shift).
Analogy: Sliding all digits of a decimal number left or right.

memcpy: Standard C function that copies a block of bytes from one memory location to another. memcpy(dest, src, n) copies exactly n bytes.
Analogy: A photocopier — duplicates a page exactly byte-for-byte.

deep copy: A copy where all nested data is duplicated. The copy is fully independent — changing the copy does not affect the original.
Analogy: Photocopying a book — you get your own copy you can write in.

shallow copy: A copy where only the top-level struct is duplicated but pointers still point to the same data. Changing the data through either copy affects both.
Analogy: Giving someone your book's index — they can find the same pages, not a new book.

exit(1): Immediately stops the entire program and returns error code 1 to the operating system. Used when something is so wrong the program cannot continue safely.
Analogy: Pulling the fire alarm — everything stops immediately.

2 — Tensor.h — Every Struct Explained

Tensor.h defines the data types — the blueprints for all tensor objects. No computation happens here; it is purely definitions.

tensorStorageId

```
typedef void *tensorStorageId;
```

A type alias for `void*`. It acts as an opaque identifier for where a tensor's data lives in storage. Think of it as a locker number — you do not need to know what is inside, just which locker to open.

■ Currently used as a placeholder for future storage backend integration. The actual data is accessed through tensor_t.data directly.

shape_t — Describes Dimensions

```
typedef struct shape {
size_t numberOfDimensions; // how many dimensions (1=vector, 2=matrix, 3=cube)
size_t *dimensions; // array: size of each dimension
size_t *orderOfDimensions; // array: traversal order (default 0,1,2,...)
} shape_t;
```

Understanding the three fields:

Field	Type	What it stores	Example (3×4 matrix)
`numberOfDimensions`	size_t	How many dimensions	2
`dimensions`	size_t*	Size of each dimension	[3, 4]
`orderOfDimensions`	size_t*	Traversal order	[0, 1] (row-major)

★ **orderOfDimensions explained:** After transposeTensor(), this array changes from [0,1] to [1,0]. No data moves — only this metadata changes. Code that loops over dimensions uses this array to know which dimension to traverse first.

sparsity_t — Sparse Tensor Marker

```
typedef enum { SPARSITY_TYPE_1, SPARSITY_TYPE_2 } sparsityType_t;
typedef struct sparsityConfig { } sparsityConfig; // empty placeholder
typedef struct sparsity {
sparsityType_t type; // which sparsity scheme
sparsityConfig *config; // future config details
} sparsity_t;
```

A sparsity mask records which weights are zero (pruned by RigL) and which are active. Storing only active weights saves memory.

■ **When sparsity pointer is NULL** → tensor is dense (all values stored). **When non-NULL** → tensor is sparse (zeros not stored). The config struct is empty for now — future versions will store mask details here.

tensor_t — THE Main Data Container

```
typedef struct tensor {
uint8_t *data; // raw bytes — the actual values
shape_t *shape; // how many dims, sizes, traversal order
quantization_t *quantization; // number format (float, int8, etc.)
sparsity_t *sparsity; // NULL = dense; non-NULL = sparse
} tensor_t;
```

Why all four fields are pointers:

Every field in tensor_t is a pointer — meaning tensor_t itself is just 16 bytes on STM32 (4 pointers × 4 bytes each). The actual data, shape arrays, and configs live elsewhere. This matters because:

- Multiple tensors can share the same **shape_t** — e.g. a weight and its gradient have identical shape.
- You can swap the **quantization_t** without touching the data buffer.
- Passing `tensor_t*` to a function is always fast — only 4 bytes transferred.

★ **Memory layout on STM32:** tensor_t (16 bytes) → shape_t (24 bytes) → dimensions[] + orderOfDimensions[] arrays → quantization_t (8 bytes) → qConfig struct → data[] buffer (N bytes, the biggest part)

parameter_t — Weight + Gradient Pair

```
typedef struct parameter {
tensor_t *param; // the learnable weight tensor
tensor_t *grad; // the gradient tensor (dLoss/dParam)
} parameter_t;
```

In training, every weight has two tensors: the weight itself and its gradient. The gradient tells the optimiser how to update the weight during backpropagation. parameter_t keeps them together so you never accidentally separate them.

✔ **param and grad always have the same shape.** calcNumberOfElementsByParameter() exploits this — it only reads param->shape.

3 — Element Counting Functions

The very first thing you need to know about any tensor is how many elements it has. Everything else — memory allocation, loops, byte calculations — depends on this.

calcNumberOfElementsByShape

```
size_t calcNumberOfElementsByShape(shape_t *shape);
```

WHAT IT DOES	Multiplies all dimension sizes together. Shape [3,4] → 3×4=12. Shape [2,3,4] → 2×3×4=24.
WHY IT EXISTS	Before allocating memory or looping, you must know the element count. Every other function calls this one.
HOW TO USE IT	Pass any shape_t* to get the total element count. Used internally by calcBytesPerTensor, copyData, printTensor.

```
shape_t *shape = ...; // say dimensions=[3,4], numberOfDimensions=2
size_t n = calcNumberOfElementsByShape(shape);
// n = 12
```

calcNumberOfElementsByTensor

```
size_t calcNumberOfElementsByTensor(tensor_t *tensor);
```

WHAT IT DOES	Calls calcNumberOfElementsByShape(tensor->shape). One extra step of indirection.
WHY IT EXISTS	Convenience wrapper — you have a tensor_t* at hand, not a shape_t*. Avoids writing tensor->shape every time.
HOW TO USE IT	Use this when you have a tensor_t* and want the element count directly.

```
tensor_t *t = ...;
size_t n = calcNumberOfElementsByTensor(t);
// same as: calcNumberOfElementsByShape(t->shape)
```

calcNumberOfElementsByParameter

```
size_t calcNumberOfElementsByParameter(parameter_t *param);
```

WHAT IT DOES	Calls calcNumberOfElementsByShape(param->param->shape). Uses the weight tensor's shape.
WHY IT EXISTS	param and grad always share a shape. This avoids the question of 'which tensor's shape do I read?'
HOW TO USE IT	Use when you have a parameter_t* and need the element count.

```
parameter_t *p = ...;
size_t n = calcNumberOfElementsByParameter(p);
// same as: calcNumberOfElementsByShape(p->param->shape)
```

4 — Size Calculation Functions

Once you know the element count, you need to know how many bytes the data occupies. This depends entirely on the quantization type — INT32 uses 4 bytes per element, BOOL uses 1 bit per element.

calcBytesPerElement

```
size_t calcBytesPerElement(quantization_t *quantization);
```

WHAT IT DOES	Returns how many bytes ONE element occupies. Switches on quantization->type.
WHY IT EXISTS	Memory allocation and pointer arithmetic need to know the element size. Centralises the type→size mapping in one place.
HOW TO USE IT	Call before allocating a data buffer or advancing a pointer by one element.

Type	Bytes per element	Note
INT32	4	sizeof(int32_t)
FLOAT32	4	sizeof(float)
SYM_INT32	4	sizeof(int32_t)
SYM	ceil(qBits/8)	1 for 8-bit, 2 for 12-bit, etc.
ASYM	ceil(qBits/8)	same rounding-up rule
BOOL	1 (misleading)	actually 1 bit — use calcNumberOfBytesForData for BOOL

calcBitsPerElement

```
size_t calcBitsPerElement(quantization_t *quantization);
```

WHAT IT DOES	Returns how many BITS one element uses. Like calcBytesPerElement but in bits.
WHY IT EXISTS	For sub-byte formats (SYM 4-bit, BOOL 1-bit), bytes-per-element loses precision. Bits are the accurate unit.
HOW TO USE IT	Used by byteConversion and calcNumberOfBytesForData to handle packed formats correctly.

calcNumberOfBytesForData — Most Important Size Function

```
size_t calcNumberOfBytesForData(quantization_t *q, size_t numberOfElements);
```

WHAT IT DOES	Returns total bytes needed for N elements of the given type. Handles BOOL packing with the (n+7)/8 ceiling-division trick.
WHY IT EXISTS	This is the function you call to know how many bytes to allocate. Direct multiplication only works for byte-aligned types.
HOW TO USE IT	Call this before reserveMemory() to know how large a data buffer to allocate.

```
// Example: allocate data buffer for a SYM_INT32 tensor of 12 elements
size_t n = calcNumberOfElementsByTensor(tensor); // 12
size_t bytes = calcNumberOfBytesForData(tensor->quantization, n); // 48
uint8_t *data = reserveMemory(bytes);
```

```
// Example: BOOL tensor with 9 elements
// 9 bits → ceil(9/8) = 2 bytes
// (9 + 7) / 8 = 2 (integer ceiling division)
size_t bytes = calcNumberOfBytesForData(boolQuant, 9); // 2
```

★ **The (n+7)/8 trick:** Integer division always truncates. 9/8 = 1 which is WRONG for 9 bits. (9+7)/8 = 16/8 = 2 which is CORRECT. General rule: ceil(a/b) = (a + b - 1) / b → for bytes: (bits + 7) / 8

calcBytesPerTensor

```
size_t calcBytesPerTensor(tensor_t *tensor);
```

WHAT IT DOES	Combines calcNumberOfElementsByTensor and calcNumberOfBytesForData into one call.
WHY IT EXISTS	Convenience — 90% of the time you want the total byte size of a whole tensor. No need to call two functions and store an intermediate result.
HOW TO USE IT	Use when you need the total byte size of a tensor_t you already have.

```
size_t total = calcBytesPerTensor(tensor);
// same as:
// size_t n = calcNumberOfElementsByTensor(tensor);
// size_t total = calcNumberOfBytesForData(tensor->quantization, n);
```

5 — BOOL Bit Operations

BOOL tensors are used as sparsity masks. They pack 8 boolean values per byte to save memory. Because of this packing you cannot use normal array indexing — you must go through tensorBoolGet and tensorBoolSet.

✔ **Why does packing matter?** A 1024-weight mask stored as uint8_t[] uses 1024 bytes. Packed BOOL uses 128 bytes — 8× smaller. On 320KB STM32 RAM this difference is significant.

tensorBoolGet — Read One Bit

```
bool tensorBoolGet(tensor_t const *tensor, size_t flatIndex);
```

WHAT IT DOES	Reads one boolean value at position flatIndex from a packed BOOL tensor. Returns true (1) or false (0).
WHY IT EXISTS	Direct array indexing gives the wrong result on packed data. This function does the byteIndex/bitIndex math correctly every time.
HOW TO USE IT	Use whenever you need to read a single mask bit. Pass the linear index of the value you want.

```
// Is weight 10 active (not pruned)?
bool isActive = tensorBoolGet(sparsityMask, 10);
if (isActive) {
// use this weight in the forward pass
}
```

Step-by-step inside the function:

Step	Code	For flatIndex=10
1 — which byte	byteIndex = 10 / 8	byteIndex = 1
2 — which bit	bitIndex = 10 % 8	bitIndex = 2
3 — shift	byte >> bitIndex	byte >> 2 → target moves to position 0
4 — isolate	& 1u	zeros all bits except position 0
5 — to bool	!= 0	1 → true, 0 → false

tensorBoolSet — Write One Bit

```
void tensorBoolSet(tensor_t *tensor, size_t flatIndex, bool value);
```

WHAT IT DOES	Writes one boolean value (true=1 or false=0) at position flatIndex. Changes only that one bit; all other bits in the byte stay untouched.
WHY IT EXISTS	During RigL grow/drop steps you need to activate or deactivate individual weights without disturbing neighbouring mask bits.
HOW TO USE IT	Call with value=true to activate a weight, value=false to prune it.

```
// Prune weight 10 (set its mask bit to 0)
tensorBoolSet(sparsityMask, 10, false);

// Grow weight 10 (activate it)
tensorBoolSet(sparsityMask, 10, true);
```

How SET vs CLEAR work:

Goal	Operation	Why it works
`Set bit to 1`	data |= (1u << bitIndex)	OR with a mask that has 1 only at the target → forces that bit to 1
`Set bit to 0`	data &= ~(1u << bitIndex)	AND with inverted mask (0 only at target) → forces that bit to 0

■ **Guard check:** tensorBoolSet checks that the tensor's quantization type is BOOL. If you accidentally call it on a FLOAT32 tensor it prints an error and calls exit(1). This prevents silently corrupting float data.

6 — Low-Level Bit Helpers

These three functions are internal building blocks for byteConversion. You will not call them directly in user code — but understanding them makes byteConversion easier to follow.

getBitmask — Create a Bit Mask

```
uint32_t getBitmask(uint32_t startbit, uint32_t endbit);
```

WHAT IT DOES	Returns a byte with 1s in bit positions [startbit, endbit) and 0s everywhere else.
WHY IT EXISTS	readByte and writeByte need a mask to isolate or protect specific bits. getBitmask generates that mask from start/end positions.
HOW TO USE IT	Internal only. Called by readByte and writeByte.

```
// Mask for bits 2,3,4 (positions 2 to 5, exclusive)
uint32_t mask = getBitmask(2, 5);
// mask = 0b00011100 = 0x1C
// bit positions: 7 6 5 4 3 2 1 0
// 0 0 0 1 1 1 0 0 ← 1s at positions 2,3,4
```

readByte — Extract Bits from a Byte

```
uint8_t readByte(uint8_t data, uint8_t startbit, uint8_t endbit);
```

WHAT IT DOES	Extracts the bits at positions [startbit, endbit) from data and returns them as a number starting at bit 0.
WHY IT EXISTS	byteConversion needs to read N-bit chunks from a packed buffer. This handles the bit-level extraction.
HOW TO USE IT	Internal only. Called inside the byteConversion loop.

```
// Read bits 2-4 of byte 0b10110100
uint8_t val = readByte(0b10110100, 2, 5);
// Step 1: mask = getBitmask(2,5) = 0b00011100
// Step 2: masked = data & mask = 0b00010100
// Step 3: shift = masked >> 2 = 0b00000101 = 5
// val = 5
```

writeByte — Insert Bits into a Byte

```
uint8_t writeByte(uint8_t existingData, uint8_t data, uint8_t startbit, uint8_t endbit);
```

WHAT IT DOES	Inserts the bits of data into existingData at positions [startbit, endbit). Other bits in existingData are not changed.
WHY IT EXISTS	byteConversion builds output bytes piece by piece, writing N-bit chunks into specific positions. writeByte handles one chunk at a time.
HOW TO USE IT	Internal only. Called inside the byteConversion loop to build output bytes.

```
// Write value 5 (0b101) into bits 2-4 of byte 0b11000011
uint8_t result = writeByte(0b11000011, 5, 2, 5);
// Step 1: shift data left: 5 << 2 = 0b00010100
// Step 2: AND with mask: 0b00010100 & 0b00011100 = 0b00010100
// Step 3: OR with existing: 0b00010100 | 0b11000011 = 0b11010111
// result = 0b11010111 (bits 2-4 changed, rest unchanged)
```

7 — byteConversion — Bit-Width Repacking

```
void byteConversion(uint8_t *dataIn, size_t dataInBits,
uint8_t *dataOut, size_t dataOutBits,
size_t numValues);
```

WHAT IT DOES	Converts a packed array of numValues values from dataInBits-wide format to dataOutBits-wide format. Handles any combination of bit widths.
WHY IT EXISTS	When quantizing weights from FLOAT32 to INT8 or INT4, the raw bit representation must be repacked. byteConversion does all the cross-byte bit arithmetic so conversion functions only need to call it with the right widths.
HOW TO USE IT	Called internally by conversion functions. E.g. convertFloatTensorToSymInt32Tensor calls byteConversion(src, 32, dst, 8, n).

Common usage patterns:

From	To	dataInBits	dataOutBits	Result
FLOAT32	INT8	32	8	Pack 32-bit floats into 8-bit integers
INT8	INT32	8	32	Expand 8-bit values back to 32-bit
INT32	INT4	32	4	Ultra-compact 4-bit packing
BOOL(1-bit)	INT8	1	8	Unpack individual bits to bytes

■ **Sliding window algorithm:** byteConversion tracks a bit-position cursor in both the input and output buffers. Each iteration copies min(bits_left_in_input_value, bits_left_in_output_byte) bits. When the input cursor crosses a byte boundary it moves to the next input byte; same for output. This handles any bit-width combination correctly.

8 — Getter and Setter Functions

Getters read a field from a struct. Setters write fields into a struct. They exist to make calling code cleaner and to hide struct internals.

getParamFromParameter / getGradFromParameter

```
tensor_t *getParamFromParameter(parameter_t *parameter);
tensor_t *getGradFromParameter (parameter_t *parameter);
```

WHAT IT DOES	Return parameter->param or parameter->grad respectively. One line each.
WHY IT EXISTS	Clean API — callers read as 'get the grad from this parameter' rather than dereferencing arrows manually. Easier to search for in large code.
HOW TO USE IT	Use whenever you have a parameter_t* and need to work with one of its tensors.

```
parameter_t *p = ...;
tensor_t *weights = getParamFromParameter(p);
tensor_t *gradient = getGradFromParameter(p);
// Apply gradient descent: weights = weights - lr * gradient
```

setTensorValues — Wire All Four Fields At Once

```
void setTensorValues(tensor_t *tensor,
uint8_t *data, shape_t *shape,
quantization_t *quantization, sparsity_t *sparsity);
```

WHAT IT DOES	Sets all four pointer fields of a tensor_t in one call.
WHY IT EXISTS	Without this you write four assignment lines every time you initialise a tensor. Error-prone if you forget one.
HOW TO USE IT	Call after allocating all components to connect everything into a working tensor.

```
tensor_t *t = reserveMemory(sizeof(tensor_t));
setTensorValues(t, dataBuffer, shapeStruct, quantStruct, NULL);
// t is now fully initialised and ready to use
```

setTensorValuesForConversion

```
void setTensorValuesForConversion(uint8_t *data, quantization_t *q,
tensor_t *originalTensor, tensor_t *outputTensor);
```

WHAT IT DOES	Wires an output tensor with new data and new quantization, but copies shape and sparsity pointers from the original.
WHY IT EXISTS	During type conversion (e.g. FLOAT32 → SYM_INT32) the output tensor has a different data buffer and different quantization, but the same shape. This function avoids manually copying those shared fields.
HOW TO USE IT	Used inside conversion functions like convertFloatTensorToSymInt32Tensor.

```
// Inside a conversion function:
uint8_t *outData = reserveMemory(outputBytes);
setTensorValuesForConversion(outData, newQuant, inputTensor, outputTensor);
// outputTensor->shape = inputTensor->shape (shared, same dimensions)
// outputTensor->sparsity = inputTensor->sparsity (shared)
// outputTensor->data = outData (new buffer)
// outputTensor->quantization = newQuant (new format)
```

setShape / setOrderOfDimsForNewTensor / setParameterValues

```
void setShape(shape_t *shape, size_t *dims, size_t n, size_t *order);
void setOrderOfDimsForNewTensor(size_t n, size_t *order);
void setParameterValues(parameter_t *p, tensor_t *param, tensor_t *grad);
```

Function	What it does	When to call
`setShape`	Wires dims, numberOfDimensions, orderOfDimensions into a shape_t	After allocating a shape_t and its arrays
`setOrderOfDimsForNewTensor`	Fills order[] with [0,1,2,...] — the default row-major order	When creating any new tensor before calling setShape
`setParameterValues`	Sets p->param and p->grad	After creating both tensors of a parameter pair

```
// Full setup sequence for a new tensor:
size_t dims[2] = {3, 4};
size_t order[2];
setOrderOfDimsForNewTensor(2, order); // order = [0,1]
shape_t *sh = reserveMemory(sizeof(shape_t));
setShape(sh, dims, 2, order);
```

9 — transposeTensor — Virtual Dimension Swap

```
void transposeTensor(tensor_t *tensor, size_t dim0Index, size_t dim1Index);
```

WHAT IT DOES	Swaps orderOfDimensions[dim0Index] with orderOfDimensions[dim1Index]. No data moves in memory.
WHY IT EXISTS	Transposing is needed in matrix operations — e.g. multiplying A by B^T. Moving all data in memory would be slow and use extra RAM. Changing only the metadata is instant and uses zero extra memory.
HOW TO USE IT	Call before a matrix multiply where you need one operand transposed.

Concrete example — transposing a 3×4 matrix:

```
// Before transpose:
// orderOfDimensions = [0, 1] → rows first (row-major, standard)
// dimensions = [3, 4] → 3 rows, 4 cols

transposeTensor(tensor, 0, 1);

// After transpose:
// orderOfDimensions = [1, 0] → cols first (column-major)
// dimensions = [3, 4] → unchanged! But now read as 4 rows, 3 cols
// Raw data bytes: unchanged! Not one byte moved.
```

★ **Virtual transpose:** The raw bytes never move. Code that iterates over elements reads orderOfDimensions to decide which dimension to advance first. After transpose, it naturally traverses in the transposed order.

10 — Print Functions

printTensor

```
void printTensor(tensor_t *tensor);
```

WHAT IT DOES	Reads every element of the tensor and prints its value. Format depends on quantization type: float for FLOAT32, integer for INT32/SYM_INT32, 0/1 for BOOL.
WHY IT EXISTS	Debugging — to inspect actual values stored in a tensor during development.
HOW TO USE IT	Call any time you want to see what a tensor contains. Avoid in production code — printf is slow and uses UART on STM32.

```
// After converting a float tensor to SYM_INT32:
printTensor(floatTensor); // prints floats before
convertTensor(floatTensor, int32Tensor);
printTensor(int32Tensor); // prints quantized integers after
```

printShape

```
void printShape(shape_t *shape);
```

WHAT IT DOES	Prints numberOfDimensions, the dimensions[] array, and the orderOfDimensions[] array.
WHY IT EXISTS	When debugging shape mismatches — e.g. a matrix multiply fails because shapes don't match.
HOW TO USE IT	Call on any shape_t* to see its current state. Useful after transposeTensor to confirm the order changed.

```
printShape(tensor->shape);
// Output:
// NumberOfDims: 2
// dimensions: 3 4
// orderOfDimensions: 0 1
```

11 — Copy Functions — Deep vs Shallow

A deep copy duplicates all data so the copy is fully independent. A shallow copy only copies pointers — both objects share the same underlying data. All copy functions here do DEEP copies.

■ **Before calling any copy function** — the destination struct must already be allocated and its pointer fields must point to allocated buffers large enough to receive the data. These functions do not allocate memory — they only copy bytes.

copyData

```
void copyData(tensor_t *dest, tensor_t *src);
```

WHAT IT DOES	Copies the raw data bytes from src to dest using memcpy. Also copies sparsity if src has one.
WHY IT EXISTS	When you need an independent duplicate of tensor values — e.g. saving a checkpoint before an update step.
HOW TO USE IT	Call as part of copyTensor (which calls copyData internally). Rarely called alone.

copyShape

```
void copyShape(shape_t *dest, shape_t *src);
```

WHAT IT DOES	Copies numberOfDimensions, dimensions[], and orderOfDimensions[] from src to dest.
WHY IT EXISTS	Shape must be copied when creating a fully independent tensor. If you only copy the pointer you would share the shape — a later transpose would affect both tensors.
HOW TO USE IT	Called inside copyTensor. Dest must have pre-allocated arrays of matching size.

copyQuantization

```
void copyQuantization(quantization_t *dest, quantization_t *src);
```

WHAT IT DOES	Copies the type tag and deep-copies the qConfig struct. For FLOAT32/BOOL (no config): sets type and NULL. For SYM_INT32: memcpy's the symInt32QConfig_t struct.
WHY IT EXISTS	If you only copy the qConfig pointer, modifying scale in one tensor would silently change the other. Deep copy makes each tensor own its scale independently.
HOW TO USE IT	Called inside copyTensor. Dest must have a pre-allocated qConfig of the right size.

copyTensor — The Top-Level Function

```
void copyTensor(tensor_t *dest, tensor_t *src);
```

WHAT IT DOES	Calls copyShape, then copyQuantization, then copyData in that order. Result: dest is a fully independent clone of src.
WHY IT EXISTS	Single entry point for tensor duplication. The order matters: copyData reads dest->shape to get the element count, so shape must be copied first.
HOW TO USE IT	Use when you need an independent copy of a tensor — e.g. saving weights before an update.

```
// Allocate destination (all components must exist first)
tensor_t *backup = reserveMemory(sizeof(tensor_t));
shape_t *bShape = reserveMemory(sizeof(shape_t));
size_t bDims[2], bOrder[2];
setShape(bShape, bDims, 2, bOrder);
quantization_t *bQuant = reserveMemory(sizeof(quantization_t));
symInt32QConfig_t *bCfg = reserveMemory(sizeof(symInt32QConfig_t));
bQuant->qConfig = bCfg;
size_t bBytes = calcBytesPerTensor(original);
uint8_t *bData = reserveMemory(bBytes);
setTensorValues(backup, bData, bShape, bQuant, NULL);

// Now copy
copyTensor(backup, original);
// backup is now a fully independent clone of original
```

12 — Complete Usage Workflow

This section shows the exact sequence of function calls needed to go from nothing to a fully set up, quantized tensor ready for training.

STEP 1 — Set up quantization config

```
// Choose SYM_INT32 quantization, 8-bit range, stochastic rounding
symInt32QConfig_t *qCfg = reserveMemory(sizeof(symInt32QConfig_t));
initSymInt32QConfigWithQMaxBits(SR_HALF_AWAY, qCfg, 8);
// qCfg->scale = 1.f (placeholder; computed during conversion)
// qCfg->roundingMode = SR_HALF_AWAY
// qCfg->qMaxBits = 8

quantization_t *quant = reserveMemory(sizeof(quantization_t));
initSymInt32Quantization(qCfg, quant);
// quant->type = SYM_INT32
// quant->qConfig = qCfg
```

STEP 2 — Set up shape

```
size_t dims[2] = {3, 4}; // 3 rows, 4 columns
size_t order[2];
setOrderOfDimsForNewTensor(2, order); // order = [0, 1]

shape_t *shape = reserveMemory(sizeof(shape_t));
setShape(shape, dims, 2, order);
```

STEP 3 — Allocate data buffer

```
size_t n = calcNumberOfElementsByShape(shape); // 12
size_t bytes = calcNumberOfBytesForData(quant, n); // 48
uint8_t *data = reserveMemory(bytes);
```

STEP 4 — Wire into a tensor

```
tensor_t *tensor = reserveMemory(sizeof(tensor_t));
setTensorValues(tensor, data, shape, quant, NULL);
mcuSimPrintStatus("after tensor alloc");
```

STEP 5 — Fill with float values, then convert to SYM_INT32

```
// You already have a FLOAT32 tensor with actual weight values
// Convert it to quantized format:
convertTensor(floatWeightTensor, tensor);
// tensor->data now contains quantized int32 values
// qCfg->scale has been computed from the actual data range
```

STEP 6 — Pair with gradient tensor as a parameter

```
// Create gradient tensor (same shape, SYM_INT32)
```

```
tensor_t *grad = ...; // same allocation sequence as above

parameter_t *param = reserveMemory(sizeof(parameter_t));

setParameterValues(param, tensor, grad);

// Access them:

tensor_t *w = getParamFromParameter(param);

tensor_t *g = getGradFromParameter(param);
```

STEP 7 — Inspect and verify

```
printShape(tensor->shape);

printTensor(tensor);

mcuSimPrintStatus("final");
```

13 — Common Mistakes and How to Avoid Them

Indexing a BOOL tensor with data[flatIndex]

■ **Problem:** data[10] reads byte 10, which holds 8 mask bits — not bool value 10.

✔ **Fix:** Always use tensorBoolGet(tensor, 10) for BOOL tensors.

Calling calcBytesPerElement for BOOL and multiplying

■ **Problem:** calcBytesPerElement returns 1 for BOOL, so 1024 * 1 = 1024 bytes — 8× too large.

✔ **Fix:** Use calcNumberOfBytesForData(quant, n) which applies (n+7)/8 for BOOL.

Copying a tensor_t by value (tensor_t copy = *original)

■ **Problem:** You copy the pointers — both tensors now share the same data, shape, and quantization. Modifying one changes the other.

✔ **Fix:** Use copyTensor() with a fully pre-allocated destination for a true deep copy.

Calling copyTensor without pre-allocating dest components

■ **Problem:** copyData calls memcpy(dest->data, ...) — if dest->data is NULL, the program crashes.

✔ **Fix:** Always allocate dest->data, dest->shape (with arrays), and dest->quantization->qConfig before calling copyTensor.

Forgetting setOrderOfDimsForNewTensor before setShape

■ **Problem:** orderOfDimensions will contain garbage values from uninitialised memory. Loops that read this array will behave unpredictably.

✔ **Fix:** Call setOrderOfDimsForNewTensor(n, order) first to fill order with [0,1,2,...].

Calling tensorBoolSet on a FLOAT32 tensor

■ **Problem:** The guard check catches this and calls exit(1), crashing the program.

✔ **Fix:** Only call tensorBoolGet/Set on tensors with quantization type BOOL.

Calling transposeTensor on a 1D tensor

■ **Problem:** The function returns immediately (guard check) — nothing happens. This is silent — no error is printed.

✔ **Fix:** Transpose only makes sense for 2D or higher. Check numberOfDimensions before calling.

14 — Quick Reference Card

Function	Input(s)	Output / Effect	When to use
`calcNumberOfElementsByShape`	shape_t*	size_t — element count	Before any size calc or loop
`calcNumberOfElementsByTensor`	tensor_t*	size_t — element count	When you have a tensor_t*
`calcNumberOfElementsByParameter`	parameter_t*	size_t — element count	When you have a parameter_t*
`calcBytesPerElement`	quantization_t*	size_t — bytes per value	Byte-aligned pointer arithmetic
`calcBitsPerElement`	quantization_t*	size_t — bits per value	Sub-byte (BOOL, SYM 4-bit) calcs
`calcNumberOfBytesForData`	quantization_t*, size_t n	size_t — total bytes	Before reserveMemory() for data
`calcBytesPerTensor`	tensor_t*	size_t — total bytes	Quick total size of a whole tensor
`tensorBoolGet`	tensor_t*, size_t idx	bool — one bit	Read one BOOL mask value
`tensorBoolSet`	tensor_t*, size_t, bool	void — sets one bit	Activate/prune one weight in mask
`getBitmask`	uint32_t s, uint32_t e	uint32_t mask	Internal only (byteConversion)
`readByte`	uint8_t, s, e	uint8_t — extracted bits	Internal only (byteConversion)
`writeByte`	existingData, data, s, e	uint8_t — merged byte	Internal only (byteConversion)
`byteConversion`	in, inBits, out, outBits, n	void — repacked buffer	Internal (conversion functions)
`getParamFromParameter`	parameter_t*	tensor_t* (weights)	Get weight tensor from param pair
`getGradFromParameter`	parameter_t*	tensor_t* (gradient)	Get gradient tensor from param pair
`setTensorValues`	tensor_t*, data, shape, q, sp	void — wires 4 fields	After allocating all components
`setTensorValuesForConversion`	data, q, orig, out	void — wires output tensor	Inside conversion functions
`setParameterValues`	parameter_t*, param, grad	void — wires pair	After creating weight + grad tensors
`setShape`	shape_t*, dims, n, order	void — wires 3 fields	After allocating shape + arrays
`setOrderOfDimsForNewTensor`	n, order[]	void — fills [0,1,2,...]	Before setShape on any new tensor
`transposeTensor`	tensor_t*, d0, d1	void — swaps order entries	Before matrix multiply needing B^T
`printTensor`	tensor_t*	void — prints all values	Debugging value inspection
`printShape`	shape_t*	void — prints dims + order	Debugging shape mismatch

`copyData`	dest tensor_t*, src tensor_t*	void — copies bytes	Part of copyTensor
`copyShape`	dest shape_t*, src shape_t*	void — copies arrays	Part of copyTensor
`copyQuantization`	dest q_t*, src q_t*	void — copies type + config	Part of copyTensor
`copyTensor`	dest tensor_t*, src tensor_t*	void — full deep copy	Save checkpoint / clone tensor